

Infosys Most Asked Java DSA Questions

(Arrays, Strings, Linked List, Stack, Queue, Trees, BST, etc)

Abhishek Rathor

Instagram id: SYNTAX ERROR (code.abhii07)

Key Features:

- 75+ Interview Questions from **Infosys** coding rounds.
- Beginner to Advanced Topics: Arrays → Strings → Linked List → Stack → Queue → Trees → BST → Advanced.
- Code in **Java** with Time & Space Complexity.
- Concept Explanations included for quick revision.
- Optimized Solutions for interviews and campus placements.

Why This Guide?

- **Save Time:** No need to search multiple sources.
- **Practice Smart:** Focus on most frequently asked questions.
- **Learn Concepts:** Step-by-step explanations for each topic.
- **Placement Ready:** Covers all **Infosys** rounds and coding challenges.

Success is not just about writing code, it's about solving problems.

Arrays

1. LeetCode #1 Two Sum

Question:

Given an array of integers `nums` and an integer `target`, return indices of the two numbers such that they add up to `target`.

Solution Logic:

Use a `HashMap` to store values and their indices for $O(n)$ lookup.

Code:

```
import java.util.HashMap;

public int[] twoSum(int[] nums, int target) {
    HashMap<Integer, Integer> map = new HashMap<>();
    for (int i = 0; i < nums.length; i++) {
        int complement = target - nums[i];
        if (map.containsKey(complement)) {
            return new int[]{map.get(complement), i};
        }
        map.put(nums[i], i);
    }
    return new int[]{-1, -1}; // No solution
}
```

Time Complexity: $O(n)$

Space Complexity: $O(n)$

2. LeetCode #121 Best Time to Buy and Sell Stock

Question:

Given an array where the i -th element is the price of a stock on day i , find the maximum profit by choosing a single day to buy and a single day to sell.

Solution Logic:

Track the minimum price so far and calculate max profit.

Code:

```
public int maxProfit(int[] prices) {
    int minPrice = Integer.MAX_VALUE;
    int maxProfit = 0;
    for (int price : prices) {
        if (price < minPrice) minPrice = price;
        else if (price - minPrice > maxProfit) maxProfit = price - minPrice;
    }
    return maxProfit;
}
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

3. LeetCode #217 Contains Duplicate

Question:

Check if the array contains any duplicates.

Solution Logic:

Use a HashSet to detect duplicates.

Code:

```
import java.util.HashSet;

public boolean containsDuplicate(int[] nums) {
    HashSet<Integer> set = new HashSet<>();
    for (int num : nums) {
        if (!set.add(num)) return true;
    }
    return false;
}
```

Time Complexity: $O(n)$

Space Complexity: $O(n)$

4. LeetCode #238 Product of Array Except Self

Question:

Return an array output such that output[i] is the product of all elements except nums[i].

Solution Logic:

Use prefix and suffix products to achieve $O(n)$ time without division.

Code:

```
public int[] productExceptSelf(int[] nums) {
    int n = nums.length;
    int[] output = new int[n];
    output[0] = 1;
    for (int i = 1; i < n; i++) {
        output[i] = output[i - 1] * nums[i - 1];
    }
    int suffix = 1;
    for (int i = n - 1; i >= 0; i--) {
        output[i] *= suffix;
        suffix *= nums[i];
    }
    return output;
}
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$ (output array not counted)

5. LeetCode #53 Maximum Subarray

Question:

Find the contiguous subarray with the largest sum.

Solution Logic:

Use Kadane's algorithm to keep track of max ending here and max so far.

Code:

```

public int maxSubArray(int[] nums) {
    int maxCurrent = nums[0], maxGlobal = nums[0];
    for (int i = 1; i < nums.length; i++) {
        maxCurrent = Math.max(nums[i], maxCurrent + nums[i]);
        maxGlobal = Math.max(maxGlobal, maxCurrent);
    }
    return maxGlobal;
}

```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

6. LeetCode #56 Merge Intervals

Question:

Given an array of intervals, merge all overlapping intervals.

Solution Logic:

Sort intervals by start time and merge overlapping ones.

Code:

```

import java.util.*;

public int[][] merge(int[][] intervals) {
    if (intervals.length <= 1) return intervals;
    Arrays.sort(intervals, (a, b) -> a[0] - b[0]);
    List<int[]> merged = new ArrayList<>();
    int[] current = intervals[0];
    merged.add(current);

    for (int[] interval : intervals) {
        if (interval[0] <= current[1]) { // Overlapping
            current[1] = Math.max(current[1], interval[1]);
        } else {
            current = interval;
            merged.add(current);
        }
    }
    return merged.toArray(new int[merged.size()][]);
}

```

Time Complexity: $O(n \log n)$ (sorting)

Space Complexity: $O(n)$

7. LeetCode #283 Move Zeroes

Question:

Move all zeroes to the end while maintaining the relative order of other elements.

Solution Logic:

Use two pointers: one to iterate and one to place non-zero elements.

Code:

```

public void moveZeroes(int[] nums) {
    int insertPos = 0;
    for (int num : nums) {
        if (num != 0) nums[insertPos++] = num;
    }
}

```

```

    }
    while (insertPos < nums.length) {
        nums[insertPos++] = 0;
    }
}

```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

8. LeetCode #36 Valid Sudoku

Question:

Determine if a 9x9 Sudoku board is valid.

Solution Logic:

Use arrays or sets to track row, column, and box constraints.

Code:

```

public boolean isValidSudoku(char[][] board) {
    for (int i = 0; i < 9; i++) {
        boolean[] row = new boolean[9];
        boolean[] col = new boolean[9];
        boolean[] box = new boolean[9];
        for (int j = 0; j < 9; j++) {
            if (board[i][j] != '.') {
                int num = board[i][j] - '1';
                if (row[num]) return false;
                row[num] = true;

                if (board[j][i] != '.') {
                    int num = board[j][i] - '1';
                    if (col[num]) return false;
                    col[num] = true;
                }

                int rowIndex = 3 * (i / 3) + j / 3;
                int colIndex = 3 * (i % 3) + j % 3;
                if (board[rowIndex][colIndex] != '.') {
                    int num = board[rowIndex][colIndex] - '1';
                    if (box[num]) return false;
                    box[num] = true;
                }
            }
        }
    }
    return true;
}

```

Time Complexity: $O(1)$ (fixed board size)

Space Complexity: $O(1)$

9. LeetCode #48 Rotate Image

Question:

Rotate an $n \times n$ 2D matrix 90 degrees clockwise in-place.

Solution Logic:

Transpose the matrix then reverse each row.

Code:

```

public void rotate(int[][] matrix) {
    int n = matrix.length;
    // Transpose
    for (int i = 0; i < n; i++) {
        for (int j = i; j < n; j++) {
            int temp = matrix[i][j];
            matrix[i][j] = matrix[j][i];
            matrix[j][i] = temp;
        }
    }
    // Reverse rows
    for (int i = 0; i < n; i++) {
        int left = 0, right = n - 1;
        while (left < right) {
            int temp = matrix[i][left];
            matrix[i][left] = matrix[i][right];
            matrix[i][right] = temp;
            left++; right--;
        }
    }
}

```

Time Complexity: $O(n^2)$

Space Complexity: $O(1)$

10. LeetCode #448 Find All Numbers Disappeared in an Array

Question:

Find all numbers in the range $[1, n]$ that do not appear in the array.

Solution Logic:

Use index marking by negating numbers.

Code:

```

import java.util.*;

public List<Integer> findDisappearedNumbers(int[] nums) {
    for (int i = 0; i < nums.length; i++) {
        int index = Math.abs(nums[i]) - 1;
        if (nums[index] > 0) nums[index] = -nums[index];
    }
    List<Integer> result = new ArrayList<>();
    for (int i = 0; i < nums.length; i++) {
        if (nums[i] > 0) result.add(i + 1);
    }
    return result;
}

```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

11. LeetCode #219 Contains Duplicate II

Question:

Check if array contains duplicates within k distance.

Solution Logic:

Use sliding window with HashSet.

Code:

```
import java.util.HashSet;

public boolean containsNearbyDuplicate(int[] nums, int k) {
    HashSet<Integer> set = new HashSet<>();
    for (int i = 0; i < nums.length; i++) {
        if (i > k) set.remove(nums[i - k - 1]);
        if (!set.add(nums[i])) return true;
    }
    return false;
}
```

Time Complexity: $O(n)$

Space Complexity: $O(k)$

12. LeetCode #350 Intersection of Two Arrays II

Question:

Return the intersection of two arrays including duplicates.

Solution Logic:

Use a HashMap to count frequencies.

Code:

```
import java.util.*;

public int[] intersect(int[] nums1, int[] nums2) {
    Map<Integer, Integer> map = new HashMap<>();
    for (int num : nums1) {
        map.put(num, map.getOrDefault(num, 0) + 1);
    }
    List<Integer> result = new ArrayList<>();
    for (int num : nums2) {
        if (map.getOrDefault(num, 0) > 0) {
            result.add(num);
            map.put(num, map.get(num) - 1);
        }
    }
    int[] res = new int[result.size()];
    for (int i = 0; i < result.size(); i++) {
        res[i] = result.get(i);
    }
    return res;
}
```

Time Complexity: $O(n + m)$

Space Complexity: $O(\min(n, m))$

13. LeetCode #66 Plus One

Question:

Given a non-empty array representing a non-negative integer, plus one to the integer.

Solution Logic:

Add one from right to left, handle carry.

Code:

```
public int[] plusOne(int[] digits) {
    for (int i = digits.length - 1; i >= 0; i--) {
        if (digits[i] < 9) {
            digits[i]++;
            return digits;
        }
        digits[i] = 0;
    }
    int[] newNumber = new int[digits.length + 1];
    newNumber[0] = 1;
    return newNumber;
}
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

14. (Skipped as duplicate of #7 Move Zeroes)

15. LeetCode #169 Majority Element

Question:

Find the majority element (appears more than $n/2$ times).

Solution Logic:

Use Boyer-Moore Voting Algorithm.

Code:

```
public int majorityElement(int[] nums) {
    int count = 0, candidate = 0;
    for (int num : nums) {
        if (count == 0) candidate = num;
        count += (num == candidate) ? 1 : -1;
    }
    return candidate;
}
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Strings

16. [LeetCode #3] Longest Substring Without Repeating Characters

Question:

Find the length of the longest substring without repeating characters.

Solution Logic:

Sliding window with HashSet or HashMap to track characters.

Code:


```

import java.util.*;

public int lengthOfLongestSubstring(String s) {
    Set<Character> set = new HashSet<>();
    int left = 0, maxLen = 0;
    for (int right = 0; right < s.length(); right++) {
        while (set.contains(s.charAt(right))) {
            set.remove(s.charAt(left));
            left++;
        }
        set.add(s.charAt(right));
        maxLen = Math.max(maxLen, right - left + 1);
    }
    return maxLen;
}

```

Time Complexity: $O(n)$

Space Complexity: $O(\min(n, \text{charset size}))$

17. [LeetCode #5] Longest Palindromic Substring

Question:

Find the longest palindromic substring in a given string.

Solution Logic:

Expand around center for each character.

Code:

```

public String longestPalindrome(String s) {
    if (s == null || s.length() < 1) return "";
    int start = 0, end = 0;
    for (int i = 0; i < s.length(); i++) {
        int len1 = expandAroundCenter(s, i, i);
        int len2 = expandAroundCenter(s, i, i + 1);
        int len = Math.max(len1, len2);
        if (len > end - start) {
            start = i - (len - 1) / 2;
            end = i + len / 2;
        }
    }
    return s.substring(start, end + 1);
}

private int expandAroundCenter(String s, int left, int right) {
    while (left >= 0 && right < s.length() && s.charAt(left) == s.charAt(right)) {
        left--;
        right++;
    }
    return right - left - 1;
}

```

Time Complexity: $O(n^2)$

Space Complexity: $O(1)$

18. [LeetCode #28] Implement strStr()

Question:

Return the index of the first occurrence of needle in haystack, or -1 if not found.

Solution Logic:

Sliding window substring check or KMP algorithm for optimization.

Code:

```
public int strStr(String haystack, String needle) {
    if (needle.isEmpty()) return 0;
    int n = haystack.length(), m = needle.length();
    for (int i = 0; i <= n - m; i++) {
        if (haystack.substring(i, i + m).equals(needle)) return i;
    }
    return -1;
}
```

Time Complexity: $O(n*m)$

Space Complexity: $O(1)$

19. [LeetCode #14] Longest Common Prefix

Question:

Find the longest common prefix among an array of strings.

Solution Logic:

Compare characters of the first string with others.

Code:

```
public String longestCommonPrefix(String[] strs) {
    if (strs == null || strs.length == 0) return "";
    for (int i = 0; i < strs[0].length(); i++) {
        char c = strs[0].charAt(i);
        for (int j = 1; j < strs.length; j++) {
            if (i == strs[j].length() || strs[j].charAt(i) != c) {
                return strs[0].substring(0, i);
            }
        }
    }
    return strs[0];
}
```

Time Complexity: $O(S)$, S = total chars in all strings

Space Complexity: $O(1)$

20. [LeetCode #344] Reverse String

Question:

Reverse a string in-place.

Solution Logic:

Swap characters from ends towards the center.

Code:

```
public void reverseString(char[] s) {
    int left = 0, right = s.length - 1;
```

```

        while (left < right) {
            char temp = s[left];
            s[left++] = s[right];
            s[right--] = temp;
        }
    }
}

```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

21. [LeetCode #125] Valid Palindrome

Question:

Check if a string is a palindrome, considering only alphanumeric characters and ignoring cases.

Solution Logic:

Use two pointers and skip non-alphanumeric chars.

Code:

```

public boolean isPalindrome(String s) {
    int left = 0, right = s.length() - 1;
    while (left < right) {
        while (left < right && !Character.isLetterOrDigit(s.charAt(left))) left++;
        while (left < right && !Character.isLetterOrDigit(s.charAt(right))) right--;
        if (Character.toLowerCase(s.charAt(left)) !=
            Character.toLowerCase(s.charAt(right))) return false;
        left++;
        right--;
    }
    return true;
}

```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

22. [LeetCode #67] Add Binary

Question:

Add two binary strings and return their sum as a binary string.

Solution Logic:

Use two pointers from end and add with carry.

Code:

```

public String addBinary(String a, String b) {
    StringBuilder sb = new StringBuilder();
    int i = a.length() - 1, j = b.length() - 1, carry = 0;
    while (i >= 0 || j >= 0 || carry == 1) {
        int sum = carry;
        if (i >= 0) sum += a.charAt(i--) - '0';
        if (j >= 0) sum += b.charAt(j--) - '0';
        sb.append(sum % 2);
        carry = sum / 2;
    }
    return sb.reverse().toString();
}

```

Time Complexity: $O(\max(m, n))$

Space Complexity: $O(\max(m, n))$

23. [LeetCode #344] Reverse String (Repeated for clarity)

Question:

Reverse string in-place (char array).

Solution Logic:

Swap from ends inward.

See question 20 above for code

24. [LeetCode #242] Valid Anagram

Question:

Check if two strings are anagrams.

Solution Logic:

Count characters and compare.

Code:

```
public boolean isAnagram(String s, String t) {
    if (s.length() != t.length()) return false;
    int[] count = new int[26];
    for (int i = 0; i < s.length(); i++) {
        count[s.charAt(i) - 'a']++;
        count[t.charAt(i) - 'a']--;
    }
    for (int c : count) {
        if (c != 0) return false;
    }
    return true;
}
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

25. [LeetCode #387] First Unique Character in a String

Question:

Find the first non-repeating character in a string and return its index.

Solution Logic:

Use a frequency array to count characters.

Code:

```
public int firstUniqChar(String s) {
    int[] count = new int[26];
    for (char c : s.toCharArray()) count[c - 'a']++;
    for (int i = 0; i < s.length(); i++) {
        if (count[s.charAt(i) - 'a'] == 1) return i;
    }
    return -1;
}
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

26. [LeetCode #58] Length of Last Word

Question:

Return the length of the last word in a string.

Solution Logic:

Traverse from the end and count characters until space.

Code:

```
public int lengthOfLastWord(String s) {
    int length = 0, i = s.length() - 1;
    while (i >= 0 && s.charAt(i) == ' ') i--;
    while (i >= 0 && s.charAt(i) != ' ') {
        length++;
        i--;
    }
    return length;
}
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

27. [LeetCode #415] Add Strings

Question:

Add two non-negative numbers represented as strings.

Solution Logic:

Add digit by digit from the end with carry.

Code:

```
public String addStrings(String num1, String num2) {
    StringBuilder sb = new StringBuilder();
    int i = num1.length() - 1, j = num2.length() - 1, carry = 0;
    while (i >= 0 || j >= 0 || carry != 0) {
        int x = i >= 0 ? num1.charAt(i--) - '0' : 0;
        int y = j >= 0 ? num2.charAt(j--) - '0' : 0;
        int sum = x + y + carry;
        sb.append(sum % 10);
        carry = sum / 10;
    }
    return sb.reverse().toString();
}
```

Time Complexity: $O(\max(m, n))$

Space Complexity: $O(\max(m, n))$

28. [LeetCode #709] To Lower Case

Question:

Convert a string to lowercase without using built-in functions.

Solution Logic:

Convert uppercase ASCII chars by adding 32.

Code:

```
public String toLowerCase(String s) {
    char[] chars = s.toCharArray();
    for (int i = 0; i < chars.length; i++) {
        if (chars[i] >= 'A' && chars[i] <= 'Z') {
            chars[i] += 32;
        }
    }
    return new String(chars);
}
```

Time Complexity: $O(n)$

Space Complexity: $O(n)$

29. [LeetCode #13] Roman to Integer**Question:**

Convert a Roman numeral to an integer.

Solution Logic:

Add values; subtract when a smaller numeral comes before a larger one.

Code:

```
import java.util.*;

public int romanToInt(String s) {
    Map<Character, Integer> map = new HashMap<>();
    map.put('I', 1);
    map.put('V', 5);
    map.put('X', 10);
    map.put('L', 50);
    map.put('C', 100);
    map.put('D', 500);
    map.put('M', 1000);
    int result = 0;
    for (int i = 0; i < s.length(); i++) {
        int value = map.get(s.charAt(i));
        if (i < s.length() - 1 && value < map.get(s.charAt(i + 1))) {
            result -= value;
        } else {
            result += value;
        }
    }
    return result;
}
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

30. [LeetCode #344] Reverse String (Repeated for emphasis)

Reverse string in-place, see question 20 above.

Linked Lists

31. [LeetCode #206] Reverse Linked List

Question:

Reverse a singly linked list.

Solution Logic:

Iteratively reverse the next pointers.

Code:

```
class ListNode {
    int val;
    ListNode next;
    ListNode(int val) { this.val = val; }
}

public ListNode reverseList(ListNode head) {
    ListNode prev = null;
    ListNode curr = head;
    while (curr != null) {
        ListNode nextTemp = curr.next;
        curr.next = prev;
        prev = curr;
        curr = nextTemp;
    }
    return prev;
}
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

32. [LeetCode #21] Merge Two Sorted Lists

Question:

Merge two sorted linked lists and return it as a new sorted list.

Solution Logic:

Use a dummy node and merge by comparing values.

Code:

```
public ListNode mergeTwoLists(ListNode l1, ListNode l2) {
    ListNode dummy = new ListNode(0);
    ListNode tail = dummy;
    while (l1 != null && l2 != null) {
        if (l1.val < l2.val) {
            tail.next = l1;
            l1 = l1.next;
        } else {
            tail.next = l2;
            l2 = l2.next;
        }
        tail = tail.next;
    }
    tail.next = (l1 != null) ? l1 : l2;
    return dummy.next;
}
```

Time Complexity: $O(n + m)$

Space Complexity: $O(1)$

33. [LeetCode #141] Linked List Cycle

Question:

Detect if a linked list has a cycle.

Solution Logic:

Use Floyd's Cycle-Finding Algorithm (fast & slow pointers).

Code:

```
public boolean hasCycle(ListNode head) {
    if (head == null) return false;
    ListNode slow = head, fast = head.next;
    while (fast != null && fast.next != null) {
        if (slow == fast) return true;
        slow = slow.next;
        fast = fast.next.next;
    }
    return false;
}
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

34. [LeetCode #142] Linked List Cycle II

Question:

Return the node where the cycle begins in a linked list or `null` if no cycle.

Solution Logic:

Detect cycle with fast/slow pointers, then find start node.

Code:

```
public ListNode detectCycle(ListNode head) {
    if (head == null) return null;
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) {
            ListNode ptr = head;
            while (ptr != slow) {
                ptr = ptr.next;
                slow = slow.next;
            }
            return ptr;
        }
    }
    return null;
}
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

35. [LeetCode #237] Delete Node in a Linked List

Question:

Delete a given node (except tail) from a singly linked list, given only access to that node.

Solution Logic:

Copy the next node's value to current and bypass next node.

Code:

```
public void deleteNode(ListNode node) {  
    node.val = node.next.val;  
    node.next = node.next.next;  
}
```

Time Complexity: $O(1)$

Space Complexity: $O(1)$

36. [LeetCode #24] Swap Nodes in Pairs

Question:

Swap every two adjacent nodes in the linked list and return its head.

Solution Logic:

Iterate and swap nodes by changing pointers.

Code:

```
public ListNode swapPairs(ListNode head) {  
    ListNode dummy = new ListNode(0);  
    dummy.next = head;  
    ListNode current = dummy;  
    while (current.next != null && current.next.next != null) {  
        ListNode first = current.next;  
        ListNode second = current.next.next;  
        first.next = second.next;  
        second.next = first;  
        current.next = second;  
        current = first;  
    }  
    return dummy.next;  
}
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

37. [LeetCode #83] Remove Duplicates from Sorted List

Question:

Remove duplicates from a sorted linked list.

Solution Logic:

Traverse and skip nodes with duplicate values.

Code:

```
public ListNode deleteDuplicates(ListNode head) {  
    ListNode current = head;
```

```

while (current != null && current.next != null) {
    if (current.val == current.next.val) {
        current.next = current.next.next;
    } else {
        current = current.next;
    }
}
return head;
}

```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

38. [LeetCode #83] Remove Duplicates from Sorted List II (Variation)

Question:

Remove all nodes that have duplicate numbers, leaving only distinct numbers.

Solution Logic:

Use dummy node and skip duplicates completely.

Code:

```

public ListNode deleteDuplicatesII(ListNode head) {
    ListNode dummy = new ListNode(0);
    dummy.next = head;
    ListNode prev = dummy;
    while (head != null) {
        if (head.next != null && head.val == head.next.val) {
            while (head.next != null && head.val == head.next.val) {
                head = head.next;
            }
            prev.next = head.next;
        } else {
            prev = prev.next;
        }
        head = head.next;
    }
    return dummy.next;
}

```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

39. [LeetCode #160] Intersection of Two Linked Lists

Question:

Find the node at which two singly linked lists intersect or `null` if no intersection.

Solution Logic:

Use two pointers that switch lists once they reach the end.

Code:

```

public ListNode getIntersectionNode(ListNode headA, ListNode headB) {
    if (headA == null || headB == null) return null;
    ListNode a = headA, b = headB;
    while (a != b) {
        a = (a == null) ? headB : a.next;
        b = (b == null) ? headA : b.next;
    }
    return a;
}

```

```

        b = (b == null) ? headA : b.next;
    }
    return a;
}

```

Time Complexity: $O(n + m)$

Space Complexity: $O(1)$

40. [LeetCode #25] Reverse Nodes in k-Group

Question:

Reverse nodes of a linked list k at a time and return its modified list.

Solution Logic:

Reverse in chunks of size k recursively or iteratively.

Code:

```

public ListNode reverseKGroup(ListNode head, int k) {
    ListNode curr = head;
    int count = 0;
    while (curr != null && count != k) {
        curr = curr.next;
        count++;
    }
    if (count == k) {
        curr = reverseKGroup(curr, k);
        while (count-- > 0) {
            ListNode temp = head.next;
            head.next = curr;
            curr = head;
            head = temp;
        }
        head = curr;
    }
    return head;
}

```

Time Complexity: $O(n)$

Space Complexity: $O(n/k)$ due to recursion stack

Stacks

41. [LeetCode #20] Valid Parentheses

Question:

Given a string containing just the characters (,), {, }, [and], determine if the input string is valid.

Solution Logic:

Use a stack to track opening brackets and check matching closing brackets.

Code:

```

public boolean isValid(String s) {
    Stack<Character> stack = new Stack<>();
    for (char c : s.toCharArray()) {
        if (c == '(') stack.push('(');
        else if (c == '{') stack.push('{');
    }
}

```

```

        else if (c == '[') stack.push('[');
        else if (stack.isEmpty() || stack.pop() != c) return false;
    }
    return stack.isEmpty();
}

```

Time Complexity: $O(n)$

Space Complexity: $O(n)$

42. [LeetCode #155] Min Stack

Question:

Design a stack that supports push, pop, top, and retrieving the minimum element in constant time.

Solution Logic:

Use two stacks: one to store all elements, another to keep track of minimums.

Code:

```

class MinStack {
    private Stack<Integer> stack;
    private Stack<Integer> minStack;

    public MinStack() {
        stack = new Stack<>();
        minStack = new Stack<>();
    }

    public void push(int val) {
        stack.push(val);
        if (minStack.isEmpty() || val <= minStack.peek()) minStack.push(val);
    }

    public void pop() {
        if (stack.peek().equals(minStack.peek())) minStack.pop();
        stack.pop();
    }

    public int top() {
        return stack.peek();
    }

    public int getMin() {
        return minStack.peek();
    }
}

```

Time Complexity: $O(1)$

Space Complexity: $O(n)$

43. [LeetCode #232] Implement Queue using Stacks

Question:

Implement a queue using two stacks.

Solution Logic:

Use one stack for enqueue, another for dequeue operations.

Code:

```

class MyQueue {
    private Stack<Integer> inStack = new Stack<>();
    private Stack<Integer> outStack = new Stack<>();

    public void push(int x) {
        inStack.push(x);
    }

    public int pop() {
        peek();
        return outStack.pop();
    }

    public int peek() {
        if (outStack.isEmpty()) {
            while (!inStack.isEmpty()) outStack.push(inStack.pop());
        }
        return outStack.peek();
    }

    public boolean empty() {
        return inStack.isEmpty() && outStack.isEmpty();
    }
}

```

Time Complexity: Amortized $O(1)$

Space Complexity: $O(n)$

44. [LeetCode #682] Baseball Game

Question:

Calculate the sum of points in a baseball game with operations +, D, and C.

Solution Logic:

Use a stack to keep track of valid rounds and apply operations accordingly.

Code:

```

public int calPoints(String[] ops) {
    Stack<Integer> stack = new Stack<>();
    for (String op : ops) {
        if (op.equals("+")) {
            int top = stack.pop();
            int newTop = top + stack.peek();
            stack.push(top);
            stack.push(newTop);
        } else if (op.equals("D")) {
            stack.push(2 * stack.peek());
        } else if (op.equals("C")) {
            stack.pop();
        } else {
            stack.push(Integer.parseInt(op));
        }
    }
    int sum = 0;
    for (int score : stack) sum += score;
    return sum;
}

```

Time Complexity: $O(n)$

Space Complexity: $O(n)$

45. [LeetCode #394] Decode String

Question:

Given an encoded string, return its decoded string.

Solution Logic:

Use two stacks to keep track of numbers and strings during decoding.

Code:

```
public String decodeString(String s) {
    Stack<Integer> counts = new Stack<>();
    Stack<String> resultStack = new Stack<>();
    StringBuilder result = new StringBuilder();
    int index = 0;

    while (index < s.length()) {
        if (Character.isDigit(s.charAt(index))) {
            int count = 0;
            while (Character.isDigit(s.charAt(index))) {
                count = count * 10 + (s.charAt(index) - '0');
                index++;
            }
            counts.push(count);
        } else if (s.charAt(index) == '[') {
            resultStack.push(result.toString());
            result = new StringBuilder();
            index++;
        } else if (s.charAt(index) == ']') {
            StringBuilder temp = new StringBuilder(resultStack.pop());
            int repeatTimes = counts.pop();
            for (int i = 0; i < repeatTimes; i++) temp.append(result);
            result = temp;
            index++;
        } else {
            result.append(s.charAt(index++));
        }
    }
    return result.toString();
}
```

Time Complexity: $O(n * k)$ (k is max repeat count)

Space Complexity: $O(n)$

46. [LeetCode #150] Evaluate Reverse Polish Notation

Question:

Evaluate the value of an arithmetic expression in Reverse Polish Notation.

Solution Logic:

Use a stack to evaluate operands and operators.

Code:

```
public int evalRPN(String[] tokens) {
    Stack<Integer> stack = new Stack<>();
```

```

for (String token : tokens) {
    if ("+".equals(token)) {
        int b = stack.pop();
        int a = stack.pop();
        stack.push(a + b);
    } else if ("-".equals(token)) {
        int b = stack.pop();
        int a = stack.pop();
        stack.push(a - b);
    } else if ("*".equals(token)) {
        int b = stack.pop();
        int a = stack.pop();
        stack.push(a * b);
    } else if ("/".equals(token)) {
        int b = stack.pop();
        int a = stack.pop();
        stack.push(a / b);
    } else {
        stack.push(Integer.parseInt(token));
    }
}
return stack.pop();
}

```

Time Complexity: $O(n)$

Space Complexity: $O(n)$

47. [LeetCode #155] Min Stack (Alternate Implementation)

Question:

Implement Min Stack using a single stack storing differences.

Solution Logic:

Store difference between current value and min to reduce space.

Code:

```

class MinStack {
    private Stack<Long> stack = new Stack<>();
    private long min;

    public void push(int x) {
        if (stack.isEmpty()) {
            stack.push(0L);
            min = x;
        } else {
            stack.push(x - min);
            if (x < min) min = x;
        }
    }

    public void pop() {
        long pop = stack.pop();
        if (pop < 0) min = min - pop;
    }

    public int top() {
        long top = stack.peek();
        if (top > 0) return (int) (top + min);
        else return (int) (min);
    }
}

```

```

    public int getMin() {
        return (int)min;
    }
}

```

Time Complexity: $O(1)$

Space Complexity: $O(n)$

48. [LeetCode #331] Verify Preorder Serialization of a Binary Tree

Question:

Verify whether a given preorder serialization of a binary tree is valid.

Solution Logic:

Use stack to simulate the tree construction.

Code:

```

public boolean isValidSerialization(String preorder) {
    String[] nodes = preorder.split(",");
    Stack<String> stack = new Stack<>();
    for (String node : nodes) {
        stack.push(node);
        while (stack.size() >= 3
            && stack.get(stack.size() - 1).equals("#")
            && stack.get(stack.size() - 2).equals("#")
            && !stack.get(stack.size() - 3).equals("#")) {
            stack.pop();
            stack.pop();
            stack.pop();
            stack.push("#");
        }
    }
    return stack.size() == 1 && stack.peek().equals("#");
}

```

Time Complexity: $O(n)$

Space Complexity: $O(n)$

Queues

49. [LeetCode #933] Number of Recent Calls

Question:

Implement a RecentCounter to count recent requests within a 3000ms window.

Solution Logic:

Use a queue to store timestamps and remove outdated calls.

Code:

```

class RecentCounter {
    private Queue<Integer> queue;

    public RecentCounter() {
        queue = new LinkedList<>();
    }
}

```



```

    public int ping(int t) {
        queue.offer(t);
        while (queue.peek() < t - 3000) {
            queue.poll();
        }
        return queue.size();
    }
}

```

Time Complexity: Amortized $O(1)$

Space Complexity: $O(n)$

50. [LeetCode #641] Design Circular Deque

Question:

Design a circular deque supporting insertion, deletion, and access from both ends.

Solution Logic:

Use an array with front and rear pointers to simulate the circular deque.

Code:

```

class MyCircularDeque {
    private int[] arr;
    private int front;
    private int rear;
    private int size;
    private int capacity;

    public MyCircularDeque(int k) {
        capacity = k;
        arr = new int[capacity];
        front = 0;
        rear = 0;
        size = 0;
    }

    public boolean insertFront(int value) {
        if (isFull()) return false;
        front = (front - 1 + capacity) % capacity;
        arr[front] = value;
        size++;
        return true;
    }

    public boolean insertLast(int value) {
        if (isFull()) return false;
        arr[rear] = value;
        rear = (rear + 1) % capacity;
        size++;
        return true;
    }

    public boolean deleteFront() {
        if (isEmpty()) return false;
        front = (front + 1) % capacity;
        size--;
        return true;
    }
}

```

```

public boolean deleteLast() {
    if (isEmpty()) return false;
    rear = (rear - 1 + capacity) % capacity;
    size--;
    return true;
}

public int getFront() {
    if (isEmpty()) return -1;
    return arr[front];
}

public int getRear() {
    if (isEmpty()) return -1;
    return arr[(rear - 1 + capacity) % capacity];
}

public boolean isEmpty() {
    return size == 0;
}

public boolean isFull() {
    return size == capacity;
}
}

```

Time Complexity: $O(1)$

Space Complexity: $O(k)$

51. [LeetCode #347] Top K Frequent Elements

Question:

Return the k most frequent elements from an array.

Solution Logic:

Use a frequency map and a min-heap priority queue to track top k elements.

Code:

```

public int[] topKFrequent(int[] nums, int k) {
    Map<Integer, Integer> count = new HashMap<>();
    for (int num : nums) count.put(num, count.getDefault(num, 0) + 1);

    PriorityQueue<Integer> heap = new PriorityQueue<>((n1, n2) -> count.get(n1) -
count.get(n2));

    for (int num : count.keySet()) {
        heap.offer(num);
        if (heap.size() > k) heap.poll();
    }

    int[] result = new int[k];
    for (int i = k - 1; i >= 0; i--) result[i] = heap.poll();
    return result;
}

```

Time Complexity: $O(n \log k)$

Space Complexity: $O(n)$

52. [LeetCode #239] Sliding Window Maximum

Question:

Find the maximum value in each sliding window of size k in an array.

Solution Logic:

Use a deque to store indexes of useful elements in the window.

Code:

```
public int[] maxSlidingWindow(int[] nums, int k) {
    if (nums == null || k <= 0) return new int[0];
    int n = nums.length;
    int[] result = new int[n - k + 1];
    Deque<Integer> deque = new LinkedList<>();

    for (int i = 0; i < nums.length; i++) {
        while (!deque.isEmpty() && deque.peekFirst() < i - k + 1) {
            deque.pollFirst();
        }
        while (!deque.isEmpty() && nums[deque.peekLast()] < nums[i]) {
            deque.pollLast();
        }
        deque.offerLast(i);
        if (i >= k - 1) {
            result[i - k + 1] = nums[deque.peekFirst()];
        }
    }
    return result;
}
```

Time Complexity: O(n)

Space Complexity: O(k)

53. [LeetCode #225] Implement Stack using Queues

Question:

Implement a stack using only queue operations.

Solution Logic:

Use two queues or one queue to simulate stack behavior.

Code (Using One Queue):

```
class MyStack {
    private Queue<Integer> queue = new LinkedList<>();

    public void push(int x) {
        queue.offer(x);
        int size = queue.size();
        while (size > 1) {
            queue.offer(queue.poll());
            size--;
        }
    }

    public int pop() {
        return queue.poll();
    }

    public int top() {
        return queue.peek();
    }
}
```

```

    }

    public boolean empty() {
        return queue.isEmpty();
    }
}

```

Time Complexity: $O(n)$ for push, $O(1)$ for pop and top

Space Complexity: $O(n)$

54. [LeetCode #933] Recent Counter (Alternate)

Question:

Same as #49, counting recent calls in a time window using queue.

Solution Logic:

Queue holds all calls, pops old calls outside time window.

Code: (Same as #49)

55. [LeetCode #622] Design Circular Queue

Question:

Implement a circular queue with fixed size.

Solution Logic:

Use an array and two pointers for front and rear, modulo arithmetic for wrap-around.

Code:

```

class MyCircularQueue {
    private int[] queue;
    private int front;
    private int rear;
    private int size;
    private int capacity;

    public MyCircularQueue(int k) {
        capacity = k;
        queue = new int[capacity];
        front = 0;
        rear = -1;
        size = 0;
    }

    public boolean enqueue(int value) {
        if (isFull()) return false;
        rear = (rear + 1) % capacity;
        queue[rear] = value;
        size++;
        return true;
    }

    public boolean dequeue() {
        if (isEmpty()) return false;
        front = (front + 1) % capacity;
        size--;
        return true;
    }
}

```

```

public int Front() {
    return isEmpty() ? -1 : queue[front];
}

public int Rear() {
    return isEmpty() ? -1 : queue[rear];
}

public boolean isEmpty() {
    return size == 0;
}

public boolean isFull() {
    return size == capacity;
}
}

```

Time Complexity: $O(1)$

Space Complexity: $O(k)$

Trees and Binary Search Trees

56. [LeetCode #94] Binary Tree Inorder Traversal

Question:

Return the inorder traversal of a binary tree's nodes' values.

Solution Logic:

Use recursion or iterative stack-based traversal.

Code (Iterative):

```

public List<Integer> inorderTraversal(TreeNode root) {
    List<Integer> result = new ArrayList<>();
    Stack<TreeNode> stack = new Stack<>();
    TreeNode current = root;

    while (current != null || !stack.isEmpty()) {
        while (current != null) {
            stack.push(current);
            current = current.left;
        }
        current = stack.pop();
        result.add(current.val);
        current = current.right;
    }
    return result;
}

```

Time Complexity: $O(n)$

Space Complexity: $O(n)$

57. [LeetCode #100] Same Tree

Question:

Check if two binary trees are the same.

Solution Logic:

Recursively check value and left/right subtrees.

Code:

```
public boolean isSameTree(TreeNode p, TreeNode q) {
    if (p == null && q == null) return true;
    if (p == null || q == null) return false;
    if (p.val != q.val) return false;
    return isSameTree(p.left, q.left) && isSameTree(p.right, q.right);
}
```

Time Complexity: $O(n)$

Space Complexity: $O(h)$ (height of tree)

58. [LeetCode #101] Symmetric Tree

Question:

Check whether a binary tree is symmetric around its center.

Solution Logic:

Use recursion to compare left subtree with right subtree mirror.

Code:

```
public boolean isSymmetric(TreeNode root) {
    return root == null || isMirror(root.left, root.right);
}

private boolean isMirror(TreeNode t1, TreeNode t2) {
    if (t1 == null && t2 == null) return true;
    if (t1 == null || t2 == null) return false;
    return (t1.val == t2.val) && isMirror(t1.left, t2.right) && isMirror(t1.right, t2.left);
}
```

Time Complexity: $O(n)$

Space Complexity: $O(h)$

59. [LeetCode #104] Maximum Depth of Binary Tree

Question:

Find the maximum depth (height) of a binary tree.

Solution Logic:

Use DFS to compute max depth recursively.

Code:

```
public int maxDepth(TreeNode root) {
    if (root == null) return 0;
    return 1 + Math.max(maxDepth(root.left), maxDepth(root.right));
}
```

Time Complexity: $O(n)$

Space Complexity: $O(h)$

60. [LeetCode #110] Balanced Binary Tree

Question:

Check if a binary tree is height-balanced.

Solution Logic:

Check height of left and right subtrees recursively and validate difference ≤ 1 .

Code:

```
public boolean isBalanced(TreeNode root) {
    return height(root) != -1;
}

private int height(TreeNode root) {
    if (root == null) return 0;
    int left = height(root.left);
    if (left == -1) return -1;
    int right = height(root.right);
    if (right == -1) return -1;
    if (Math.abs(left - right) > 1) return -1;
    return Math.max(left, right) + 1;
}
```

Time Complexity: $O(n)$

Space Complexity: $O(h)$

61. [LeetCode #111] Minimum Depth of Binary Tree

Question:

Find the minimum depth of a binary tree (shortest distance from root to a leaf).

Solution Logic:

Use DFS considering only paths ending at leaf nodes.

Code:

```
public int minDepth(TreeNode root) {
    if (root == null) return 0;
    if (root.left == null) return 1 + minDepth(root.right);
    if (root.right == null) return 1 + minDepth(root.left);
    return 1 + Math.min(minDepth(root.left), minDepth(root.right));
}
```

Time Complexity: $O(n)$

Space Complexity: $O(h)$

62. [LeetCode #112] Path Sum

Question:

Check if the tree has a root-to-leaf path such that adding all values equals a given sum.

Solution Logic:

DFS subtracting node values from sum until leaf node.

Code:

```
public boolean hasPathSum(TreeNode root, int sum) {
    if (root == null) return false;
    if (root.left == null && root.right == null) return sum == root.val;
}
```

```

        return hasPathSum(root.left, sum - root.val) || hasPathSum(root.right, sum -
root.val);
    }

```

Time Complexity: $O(n)$

Space Complexity: $O(h)$

63. [LeetCode #114] Flatten Binary Tree to Linked List

Question:

Flatten a binary tree to a linked list in-place following preorder traversal.

Solution Logic:

Recursively flatten left and right subtrees and re-link nodes.

Code:

```

public void flatten(TreeNode root) {
    if (root == null) return;
    flatten(root.left);
    flatten(root.right);

    TreeNode tempRight = root.right;
    root.right = root.left;
    root.left = null;

    TreeNode current = root;
    while (current.right != null) {
        current = current.right;
    }
    current.right = tempRight;
}

```

Time Complexity: $O(n)$

Space Complexity: $O(h)$

64. [LeetCode #116] Populating Next Right Pointers in Each Node

Question:

Populate each next pointer to point to its next right node in a perfect binary tree.

Solution Logic:

Use level order traversal or constant space approach.

Code (Constant Space):

```

public void connect(TreeLinkNode root) {
    if (root == null) return;

    TreeLinkNode leftmost = root;

    while (leftmost.left != null) {
        TreeLinkNode head = leftmost;

        while (head != null) {
            head.left.next = head.right;
            if (head.next != null) {
                head.right.next = head.next.left;
            }
        }
    }
}

```



```

        head = head.next;
    }
    leftmost = leftmost.left;
}
}

```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

65. [LeetCode #98] Validate Binary Search Tree

Question:

Determine if a binary tree is a valid BST.

Solution Logic:

Use recursion with min and max bounds for node values.

Code:

```

public boolean isValidBST(TreeNode root) {
    return validate(root, null, null);
}

private boolean validate(TreeNode node, Integer low, Integer high) {
    if (node == null) return true;
    if ((low != null && node.val <= low) || (high != null && node.val >= high)) return false;
    return validate(node.left, low, node.val) && validate(node.right, node.val, high);
}

```

Time Complexity: $O(n)$

Space Complexity: $O(h)$

Bonus: Advanced / Mixed Questions

66. [LeetCode #139] Word Break

Question:

Given a string and a dictionary of words, determine if the string can be segmented into a space-separated sequence of dictionary words.

Solution Logic:

Use dynamic programming to check substring partitions.

Code:

```

public boolean wordBreak(String s, List<String> wordDict) {
    Set<String> wordSet = new HashSet<>(wordDict);
    boolean[] dp = new boolean[s.length() + 1];
    dp[0] = true;

    for (int i = 1; i <= s.length(); i++) {
        for (int j = 0; j < i; j++) {
            if (dp[j] && wordSet.contains(s.substring(j, i))) {
                dp[i] = true;
                break;
            }
        }
    }
}

```

```
        return dp[s.length()];
    }
}
```

Time Complexity: $O(n^2)$

Space Complexity: $O(n)$

67. [LeetCode #198] House Robber

Question:

Find the maximum amount of money you can rob without robbing two adjacent houses.

Solution Logic:

Use DP to track max money at each step.

Code:

```
public int rob(int[] nums) {
    if (nums == null || nums.length == 0) return 0;
    int prev = 0, curr = 0;

    for (int num : nums) {
        int temp = curr;
        curr = Math.max(prev + num, curr);
        prev = temp;
    }
    return curr;
}
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

68. [LeetCode #200] Number of Islands

Question:

Count the number of islands (connected groups of '1's) in a 2D grid.

Solution Logic:

Use DFS or BFS to mark visited islands.

Code:

```
public int numIslands(char[][] grid) {
    int count = 0;
    for (int i = 0; i < grid.length; i++) {
        for (int j = 0; j < grid[0].length; j++) {
            if (grid[i][j] == '1') {
                dfs(grid, i, j);
                count++;
            }
        }
    }
    return count;
}

private void dfs(char[][] grid, int i, int j) {
    if (i < 0 || j < 0 || i >= grid.length || j >= grid[0].length || grid[i][j] == '0')
        return;
    grid[i][j] = '0';
    dfs(grid, i + 1, j);
}
```

```

        dfs(grid, i - 1, j);
        dfs(grid, i, j + 1);
        dfs(grid, i, j - 1);
    }

```

Time Complexity: $O(mn)$

Space Complexity: $O(mn)$

69. [LeetCode #215] Kth Largest Element in an Array

Question:

Find the kth largest element in an unsorted array.

Solution Logic:

Use a min-heap (priority queue) of size k.

Code:

```

public int findKthLargest(int[] nums, int k) {
    PriorityQueue<Integer> pq = new PriorityQueue<>();
    for (int num : nums) {
        pq.offer(num);
        if (pq.size() > k) {
            pq.poll();
        }
    }
    return pq.peek();
}

```

Time Complexity: $O(n \log k)$

Space Complexity: $O(k)$

70. [LeetCode #239] Sliding Window Maximum

Question:

Find the maximum in each sliding window of size k in an array.

Solution Logic:

Use a deque to maintain max candidates.

Code:

```

public int[] maxSlidingWindow(int[] nums, int k) {
    if (nums == null || k <= 0) return new int[0];
    int n = nums.length;
    int[] result = new int[n - k + 1];
    Deque<Integer> deque = new LinkedList<>();

    for (int i = 0; i < nums.length; i++) {
        while (!deque.isEmpty() && deque.peekFirst() < i - k + 1) {
            deque.pollFirst();
        }
        while (!deque.isEmpty() && nums[deque.peekLast()] < nums[i]) {
            deque.pollLast();
        }
        deque.offerLast(i);
        if (i >= k - 1) {
            result[i - k + 1] = nums[deque.peekFirst()];
        }
    }
}

```

```

    }
    return result;
}

```

Time Complexity: $O(n)$

Space Complexity: $O(k)$

71. [LeetCode #329] Longest Increasing Path in a Matrix

Question:

Find the longest strictly increasing path in a matrix.

Solution Logic:

Use DFS + memoization to avoid recomputation.

Code:

```

public int longestIncreasingPath(int[][] matrix) {
    if (matrix == null || matrix.length == 0) return 0;
    int m = matrix.length, n = matrix[0].length;
    int[][] memo = new int[m][n];
    int max = 0;

    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            max = Math.max(max, dfs(matrix, i, j, memo));
        }
    }
    return max;
}

private int dfs(int[][] matrix, int i, int j, int[][] memo) {
    if (memo[i][j] != 0) return memo[i][j];
    int m = matrix.length, n = matrix[0].length;
    int max = 1;
    int[] dx = {0, 0, 1, -1};
    int[] dy = {1, -1, 0, 0};

    for (int d = 0; d < 4; d++) {
        int x = i + dx[d], y = j + dy[d];
        if (x >= 0 && y >= 0 && x < m && y < n && matrix[x][y] > matrix[i][j]) {
            max = Math.max(max, 1 + dfs(matrix, x, y, memo));
        }
    }
    memo[i][j] = max;
    return max;
}

```

Time Complexity: $O(mn)$

Space Complexity: $O(mn)$

72. [LeetCode #347] Top K Frequent Elements

Question:

Return the k most frequent elements in an array.

Solution Logic:

Use a frequency map + max heap or bucket sort.

Code:

```
public int[] topKFrequent(int[] nums, int k) {
    Map<Integer, Integer> freq = new HashMap<>();
    for (int num : nums) {
        freq.put(num, freq.getOrDefault(num, 0) + 1);
    }

    PriorityQueue<Integer> pq = new PriorityQueue<>((a, b) -> freq.get(a) - freq.get(b));
    for (int num : freq.keySet()) {
        pq.offer(num);
        if (pq.size() > k) {
            pq.poll();
        }
    }

    int[] result = new int[k];
    for (int i = k - 1; i >= 0; i--) {
        result[i] = pq.poll();
    }
    return result;
}
```

Time Complexity: $O(n \log k)$

Space Complexity: $O(n)$

73. [LeetCode #378] Kth Smallest Element in a Sorted Matrix

Question:

Find the kth smallest element in a sorted matrix (rows and columns sorted).

Solution Logic:

Use min-heap or binary search.

Code (Min-Heap):

```
public int kthSmallest(int[][] matrix, int k) {
    int n = matrix.length;
    PriorityQueue<int[]> pq = new PriorityQueue<>((a,b) -> matrix[a[0]][a[1]] - matrix[b[0]][b[1]]);

    for (int i = 0; i < n; i++) {
        pq.offer(new int[]{i, 0});
    }

    int[] curr = new int[2];
    for (int i = 0; i < k; i++) {
        curr = pq.poll();
        if (curr[1] < n - 1) {
            pq.offer(new int[]{curr[0], curr[1] + 1});
        }
    }
    return matrix[curr[0]][curr[1]];
}
```

Time Complexity: $O(k \log n)$

Space Complexity: $O(n)$

74. [LeetCode #394] Decode String

Question:

Decode strings with nested encoded patterns like "3[a2[c]]".

Solution Logic:

Use stacks to decode.

Code:

```
public String decodeString(String s) {
    Stack<Integer> counts = new Stack<>();
    Stack<StringBuilder> resultStack = new Stack<>();
    StringBuilder current = new StringBuilder();
    int index = 0;

    while (index < s.length()) {
        char c = s.charAt(index);

        if (Character.isDigit(c)) {
            int count = 0;
            while (Character.isDigit(s.charAt(index))) {
                count = count * 10 + (s.charAt(index) - '0');
                index++;
            }
            counts.push(count);
        } else if (c == '[') {
            resultStack.push(current);
            current = new StringBuilder();
            index++;
        } else if (c == ']') {
            StringBuilder temp = current;
            current = resultStack.pop();
            int repeatTimes = counts.pop();
            for (int i = 0; i < repeatTimes; i++) {
                current.append(temp);
            }
            index++;
        } else {
            current.append(c);
            index++;
        }
    }
    return current.toString();
}
```

Time Complexity: $O(n)$

)

Space Complexity: $O(n)$

75. [LeetCode #721] Accounts Merge

Question:

Merge accounts where multiple emails belong to the same user.

Solution Logic:

Use Union-Find (Disjoint Set Union) to merge emails.

Code:

```

public List<List<String>> accountsMerge(List<List<String>> accounts) {
    Map<String, String> emailToName = new HashMap<>();
    Map<String, String> parent = new HashMap<>();

    for (List<String> account : accounts) {
        String name = account.get(0);
        for (int i = 1; i < account.size(); i++) {
            parent.put(account.get(i), account.get(i));
            emailToName.put(account.get(i), name);
        }
    }

    for (List<String> account : accounts) {
        String firstEmail = account.get(1);
        for (int i = 2; i < account.size(); i++) {
            union(parent, firstEmail, account.get(i));
        }
    }

    Map<String, TreeSet<String>> unions = new HashMap<>();
    for (String email : parent.keySet()) {
        String root = find(parent, email);
        unions.computeIfAbsent(root, k -> new TreeSet<>()).add(email);
    }

    List<List<String>> result = new ArrayList<>();
    for (String root : unions.keySet()) {
        List<String> emails = new ArrayList<>(unions.get(root));
        emails.add(0, emailToName.get(root));
        result.add(emails);
    }

    return result;
}

private String find(Map<String, String> parent, String s) {
    while (!parent.get(s).equals(s)) {
        parent.put(s, parent.get(parent.get(s)));
        s = parent.get(s);
    }
    return s;
}

private void union(Map<String, String> parent, String a, String b) {
    String rootA = find(parent, a);
    String rootB = find(parent, b);
    if (!rootA.equals(rootB)) {
        parent.put(rootA, rootB);
    }
}

```

Time Complexity: $O(A * \alpha(A))$ where A is total emails, α is inverse Ackermann function

Space Complexity: $O(A)$

Congratulations!

You have successfully completed this DSA & Tech guide. This marks a step forward in your coding journey! Remember, learning never stops—keep practicing, solving problems, and building projects. Every line of code you write brings you closer to becoming a master of Data Structures, Algorithms, and modern tech skills.

Stay connected with us for more coding tutorials, challenges, tips, and tech content on Instagram: [@SYNTAXERROR](#). Let's grow together as a community of passionate coders!

© 2025-26 SYNTAX ERROR. All Rights Reserved.

This content is exclusively created by SYNTAX ERROR. Unauthorized copying, sharing, or redistribution of this DSA & Tech content is strictly prohibited. Protecting intellectual property ensures quality learning for everyone and maintains the integrity of this work.